AUE-CSC 204 – WEB TECHNOLOGIES
LECTURE 3

Mr. Jeremiah Ademola BALOGUN

Department of Computer Science

+2349063062341

Balogun.Jeremiah@adelekeuniversity.edu.ng

The Skeleton & Semantic Web

Now, we begin building.

- In web development, **HyperText Markup Language (HTML)** is the skeleton of your application.
  - Without it, there is no structure.

- However, in 2026, we don't just write "any" HTML.
  - We write **Semantic HTML.**

What is "Semantics"?

- In linguistics, semantics is the study of meaning.
  - In HTML, a Semantic Element is a tag that clearly describes its content to both the browser and the developer.

  - **Non-Semantic Elements**: and .
    - They tell us nothing about what is inside them.
    - They are just empty boxes.

  - **Semantic Elements**: <header>, <article>, and <footer>.
    - They tell the browser exactly what their role is.



Why Semantics Matter (The "Why" for Your Major)

- **For the Computer Scientist (SEO):** Search engines like Google use "web crawlers" to index your site.
  - If you use a <nav> tag for your menu, the crawler immediately knows where your important links are. This helps your site rank higher in search results.

- **For the Cyber Security Student (Accessibility):** Visually impaired users use "Screen Readers" to browse the web.
  - A screen reader can "jump" directly to the <main> content or skip the <nav> only if you use semantic tags.
  - In many countries, accessibility is a **legal requirement**.

- **For the Software Engineer (Maintainability):** Semantic code is easier to read.
  - When you return to a project after 6 months, seeing <section id="results"> is much clearer than seeing



The Key HTML5 Structural Elements

Tag	Purpose
<header>	Introductory content, logos, and the main title.
<nav>	A block of navigation links (menus).
<main>	The unique, central content of the page (only one per page!).
<section>	A thematic grouping of content, usually with its own heading.
<article>	Independent, self-contained content (like a blog post or news story).
<aside>	Tangentially related content (sidebars, ads, or "quick facts").
<footer>	Copyright info, contact details, and site maps.

Local Context: Building "Lightweight" and Meaningful

- In Nigeria, mobile data is a precious resource.
- Semantic HTML is often **lighter** than "div-heavy" code because you don't need to add as many class or id attributes to explain what the code is doing.
- By using the "right tag for the right job," you create cleaner code that loads faster on poor networks.
-
- Semantic and Non-Semantic HTML5 Elements
  - This video provides a quick visual comparison to help you distinguish between meaningful tags and generic containers in modern web design.

Student Interaction: The "Semantic Refactor"

- **The Scenario:** You find this code in an old project for a local hospital:

```
<div id="top-part">
  <div class="logo">City Hospital</div>
  <div class="menu">Home | Appointments | Contact</div>
</div>
```

- The Task:
  - Identify the problem: Why is this "non-semantic"?
  - Refactor it:
    - How would you rewrite this using the new tags we learned today?
    - **Hint**: Think about using <header> and <nav>.

CSS3 – The Skin & The Box Model

Next, we add the "skin."

Cascading Style Sheets (CSS) is what transforms a plain, boring document into a professional, visually engaging experience.

- For Computer Science and Software Engineering students, CSS is about the Separation of Concerns.

- For Cyber Security students, CSS is a vital layer to understand; not just for UI, but because malicious styles (like hidden "clickjacking" layers) are a real threat.

CSS Syntax: The "Command" Structure

- To style an element, you need three things: a Selector (who), a Property (what), and a Value (how much).
  - **Selector**: Targets the HTML element (e.g., h1, .btn-success, #main-header).
  - **Property**: The feature you want to change (e.g., color, font-size, margin).
  - **Value**: The specific setting (e.g., blue, 24px, 10px).

CSS Selectors: The "Search" Tool

- As your projects grow, you need to be precise.
- You don't want to style *every* paragraph; you might only want to style the "Alert" paragraph.

Selector Type	Example	Use Case
Element	p { ... }	Apply a base style to all paragraphs.
Class	.btn { ... }	Style multiple elements (like all buttons) the same way.
ID	#logo { ... }	Style **one unique** element (like the site logo).
Pseudo-class	a:hover { ... }	Change the style when a user hovers their mouse over a link.



The "Golden Rule": The CSS Box Model

- This is the most important concept in web layout.
  - In CSS, everything is a box.
  - Understanding how these boxes are sized is the difference between a professional site and a broken one.
    - **Content**: The actual text or image.
    - **Padding**: The "inner space" between the content and the border. (Makes buttons look "breathable").
    - **Border**: The line surrounding the padding and content.
    - **Margin**: The "outer space" between this box and its neighbors. (Prevents elements from sticking together).

- **The "Lagos Traffic" Analogy**:
  - Think of the Content as the people in a car, the Padding as the interior space of the car, the Border as the car's body, and the Margin as the "social distancing" space between your car and the one in front of you in traffic!

Local Relevance: Speed & Caching

- In Nigeria, where data costs are a concern and networks can be slow (2G/3G in some areas), CSS provides a massive advantage: Caching.
  - **Efficiency**: Browsers download your .css file once and "remember" it.
  - **Global Context**: By using an external stylesheet, you make your website significantly faster than if you put styles directly in every HTML tag.
    - This saves your users' data and makes your app feel "snappy."



Typography & Colors

- **Fonts:** We use font-family.
  - *Note:* Always provide a "fallback" like sans-serif in case a custom font doesn't load.

- **Colors:** You can use names (red), Hex codes (#FF0000), or RGB (rgb(255, 0, 0)).

- **Pro-Tip:** Ensure high contrast (e.g., black text on a white background) for accessibility.



Student Interaction: The "Box Model" Debugging

- The Task: Open the project you started.
  - Add a <style> block in your <head>.
  - Give your name (h1) a border of 2px solid green.
  - Add padding: 20px; and margin: 50px;.
  - **Observe**: See how the "green box" moves away from the edge (margin) and how the text gets space inside the box (padding).

Layouts – Mastering CSS Flexbox

- Up until now, we've been styling individual "boxes." Today, we learn how to arrange those boxes into professional layouts.
- Before 2012, web layout was a nightmare of "floats" and "table hacks."
- Today, we have Flexbox (Flexible Box Layout); a one-dimensional layout model that makes aligning elements easy.

The Flex Concept: Parent and Child

- Flexbox works on a **Parent-Child** relationship.
  - **Flex Container**: The parent element where you set display: flex;.
  - **Flex Items**: The immediate children of that container.
- Once you enable flex, you gain control over the Main Axis (the direction items go) and the Cross Axis (perpendicular to the main axis).



Core Container Properties

- To master Flexbox, you only need to learn a few key properties that sit on the **Parent**:

Property	What it does	Common Values
flex-direction	Sets the direction of items.	row (default), column, row-reverse
justify-content	Aligns items on the **Main Axis**.	flex-start, center, space-between, space-around
align-items	Aligns items on the **Cross Axis**.	flex-start, center, stretch (default)
flex-wrap	Allows items to wrap to a new line.	nowrap (default), wrap



Local Context: The "Responsive Mobile Navbar"

- In Nigeria, over **80% of web traffic** comes from mobile devices.
  - A website that looks great on a desktop but breaks on a phone is a failure.
- **The Scenario:** You are building a navigation bar for a Nigerian Fintech app. On a desktop, the logo and links should be side-by-side.
  - On a small phone, they might need to stack or space out differently.
- **The Flex Solution:**

```
.navbar {
  display: flex;
  justify-content: space-between; /* Logo on left, Links on right */
  align-items: center;            /* Perfectly centered vertically */
  padding: 10px 20px;
}
```

Why Flexbox is a "Data-Saver"

- For the **Software Engineer**, Flexbox is efficient.
  - It requires much less code than older layout methods.

- For the **Cyber Security** student, clean code is easier to audit.
  - Because Flexbox handles the spacing automatically, you don't need complex "hacks" that might cause UI bugs or layout shifts on slow Nigerian networks.



Student Interaction: The "Center Everything" Challenge

- One of the oldest jokes in web development is how hard it is to center a div.
  - With Flexbox, it’s only three lines of code.
- The Task: Create a container in your HTML and try this:

```css
.parent {
  display: flex;
  justify-content: center; /* Horizontal centering */
  align-items: center;     /* Vertical centering */
  height: 100vh;           /* Take full screen height */
}
```

- Learn CSS Flexbox in 20 Minutes
  - This video provides a practical, visual guide to mastering the axes and alignment properties we discussed today.